Decision Orchestration: Learning Annotation Procedures from a Handful of Gold Documents

Anonymous ACL submission

Abstract

Every new annotation scheme starts with a guideline and almost no data. Supervised parsers need thousands of examples, and an LLM prompted with the guideline ignores its conventions, emits malformed output, and cannot be debugged. We observe that a guideline is mostly a procedure: most steps are deterministic, and only a few call for judgment. We propose *decision orchestration*, which compiles a guideline into an executable procedure. Code runs the steps, enforces well-formedness and assembles the output, and a model is consulted only at decision points, for a schema-constrained fragment or for probabilities over the options of a closed question. We then learn the procedure itself, not model weights, from a handful of gold documents. Decision thresholds are calibrated by measurement, uncertain decisions are routed to a stronger model, and reflection attributes each error to the node that made it and keeps an edit only if it measurably helps. On Chinese Uniform Meaning Representation parsing, orchestration lifts a 9B model from 0 to 17.6 F, two gold documents raise a frontier model from 45.3 to 52.6 F, recalibration on four documents adapts a new decision model (+2.9 F), and routing recovers 75% of a stronger model's gain at 26% of its cost. [TODO: PARSEME sentence once E9 is run]

1 Introduction

New annotation schemes appear all the time: an intent taxonomy for a product, an event schema for a news domain, a coding scheme in political science, a meaning representation for an under-documented language. Each starts with a guideline and almost no annotated data, and producing the first data is the bottleneck. Annotators must learn the guideline and label item after item before a model can be trained to take over (Settles, 2009). For rich semantic representations the cost is extreme. Uniform Meaning Representation (UMR; Van Gysel et al., 2021) requires a full graph for every sentence and a document-level layer of temporal, modal and coreference relations, and after years of effort its release has only a few hundred gold graphs per major language (Bonn et al., 2025).

LLMs promise to shortcut this cold start, and for labels from a familiar inventory they often match crowd workers (Wang et al., 2021; Gilardi et al., 2023; Tan et al., 2024). Structured annotation is harder. Asked for a whole UMR or AMR graph, LLMs ignore the conventions the guideline defines, produce malformed graphs, and entangle their errors so that no one can tell which part of the guideline failed (Ettinger et al., 2023; Sun et al., 2024). In our experiments a frontier model given the sentence and the guideline reaches about 30 sentence F, and an open 9B model produces no graph that can be scored at all. Training on annotated data is what the cold start lacks. Agent harnesses that let the model plan its own steps (Yao et al., 2023) leave cost and output format to the model, hide which decision caused an error, and work only with the strongest models.

Our starting point is that *a guideline is mostly a procedure*. To annotate a sentence in UMR, one segments it into clauses, finds each predicate, looks up its frame, attaches arguments, writes names and dates in fixed patterns, merges mentions of one referent, keeps the graph well-formed and every word covered, and finally relates each event to time, to its source and to earlier mentions. Most of these steps can be executed by code. A few are judgments that no rule captures: is this word an event here, which sense is meant, is the aspect a state or a performance, does this mention refer to that one?

We propose **decision orchestration** (Figure 1). The guideline is compiled into a registry of *skills* that a deterministic harness executes. *Rule nodes* are code. *Decision nodes* consult a model, and the model only decides. It either fills a schema-constrained fragment that the harness assembles,

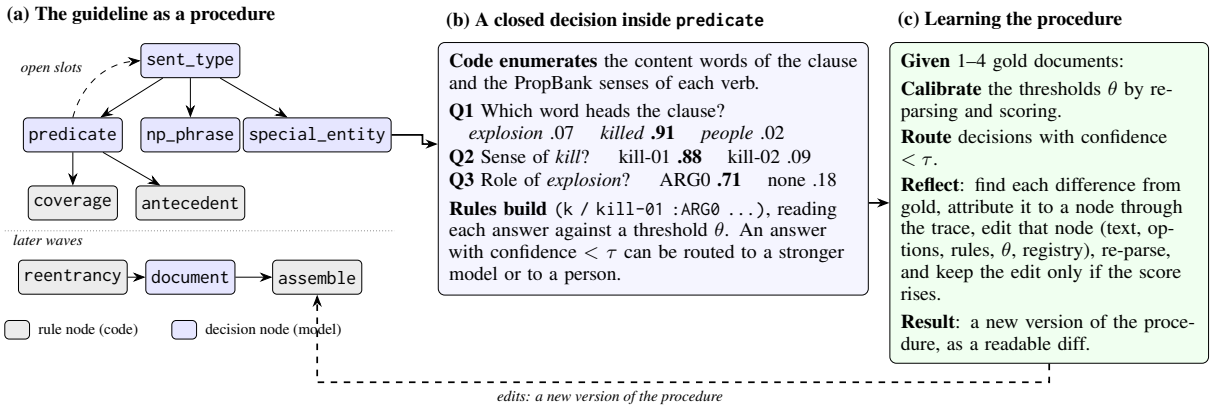


Figure 1: Decision orchestration. (a) The guideline is compiled into skills dispatched by a registry. Rule nodes are code; decision nodes consult a model. Shown is part of the UMR procedure (all nodes: Table 8, appendix). (b) In closed mode, a decision node asks questions whose options code has enumerated, and rules read the answer probabilities. (c) With a handful of gold documents, the procedure itself is learned. Example sentence, in English for readability: *Police said the explosion killed two people.*

or it answers *closed questions* (yes/no, or a choice among candidates that code enumerated) with a probability per option, which rules turn into structure. The model never chooses the next step or writes the final format, and every decision is recorded with its input. The same interface can be served by a frontier LLM, a small open model, or a *decision model*, a service that answers closed questions with probabilities and generates no text (TypeSafe, 2026; OpenAI, 2026; Microsoft, 2026).

The procedure, not a model, then becomes the thing to learn. Because it is explicit and every decision is attributable, a handful of gold documents is enough to improve it, without updating any weights (§4). **Calibration** sets the thresholds through which rules read decision probabilities by measurement on a few documents, which is needed whenever the decision model changes. **Routing** sends the few uncertain decisions to a stronger model or, in a cold start, to a person. **Reflection** has an LLM compare the output with gold, follow the execution trace to the node that caused each difference, and edit that node: its instructions, candidate options, assembly rules, thresholds, or the registry. When parsing is cheap, each edit is a controlled trial on training, regression and development documents, kept only if it helps. Every learned change is a readable diff.

On Chinese UMR parsing (sentence graphs and document-level relations), learning from one to four documents (51–201 sentences), we find that (i) orchestration makes structured annotation feasible for small models: Qwen3.5-9B goes from 0 to 17.6 F, a 9B decision model reaches 30.8, and a 27B open model reaches 94% of a frontier model; (ii) the procedure learns from very little data: two gold documents raise a frontier model from 45.3 to 52.6 F, and the same loop improves a decision-model procedure on documents it never saw; (iii) decisions can be calibrated: recalibrating on four documents gives a new decision model +2.9 F, and routing low-confidence decisions recovers 75% of a frontier model's gain at 26% of its cost; and (iv) errors stay debuggable: lost credit is attributed to skills, measured reflection rejects 40% of its own proposals, and swapping one component at a time locates the bottleneck. We also show where the approach stops paying off, since gains on unseen documents are an order of magnitude smaller than on training documents. [TODO: one sentence on the second benchmark (E9)]

Our contributions are **decision orchestration**, a recipe for compiling a guideline into a procedure in which code owns control flow and well-formedness and the model only decides; **a way to learn the procedure from a handful of gold documents** by calibration, routing, and attributed, measured reflection, with no weight updates; and **an empirical study** across open and frontier LLMs and three decision-model services of what the procedure learns, what transfers, and what it costs.

2 Related Work

LLMs as annotators. LLMs can replace or assist crowd workers on many labeling tasks (Wang et al., 2021; Gilardi et al., 2023; He et al., 2024; Ziems

et al., 2024; Tan et al., 2024). Human–LLM collaboration allocates items by uncertainty (Li et al., 2023) or has people verify LLM labels (Wang et al., 2024). These works mostly produce one label per item; we target structured annotation and route single decisions within an item. GoLLIE (Sainz et al., 2024) fine-tunes models to follow guidelines written as code, Code4Struct (Wang et al., 2023) casts event structures as code, and guideline learning (Pang et al., 2023) induces guideline text from errors. There the model reads the guideline; here the harness executes it, and learning edits its procedure, options, rules and thresholds, not only its text.

LM programs and structured output. Tasks are often decomposed into several LM calls: prompt chains (Wu et al., 2022), decomposed prompting (Khot et al., 2023), programs that call models (Gao et al., 2023; Gupta and Kembhavi, 2023), and declarative LM programs (Beurer-Kellner et al., 2023; Khattab et al., 2024). Constrained decoding makes one output follow a grammar (Willard and Louf, 2023; Geng et al., 2023), although restricting the format of a whole answer can hurt reasoning (Tam et al., 2024). Agents (Yao et al., 2023) leave control flow to the model. Decision orchestration is at the other end. Control flow is a registry dispatched on typed decisions, rules are first-class nodes, each call returns one small decision, and closed decisions return distributions.

Optimizing prompts, programs and workflows. Prompts can be optimized by search (Zhou et al., 2023), by textual gradients (Pryzant et al., 2023; Yuksekgonul et al., 2025), by LLMs as optimizers (Yang et al., 2024), or jointly with demonstrations across modules (Opsahl-Ong et al., 2024). GEPA (Agrawal et al., 2026) evolves prompts by reflecting on traces, and ACE (Zhang et al., 2026) grows a playbook from feedback. Agents can distill insights or rules from experience (Zhao et al., 2024; Zhu et al., 2023). Whole workflows can be searched (Zhang et al., 2025; Hu et al., 2025), and a scaffold can improve itself (Zelikman et al., 2024). Self-feedback can help (Madaan et al., 2023; Shinn et al., 2023), but without external feedback self-correction is unreliable (Huang et al., 2024). Our reflection is closest to GEPA and ACE, but it differs in three ways. It attributes each error to one node through the trace and the scorer's alignment. It edits code (options, rules, thresholds, registry) as well as text. And it accepts each edit only after a measured trial on a handful of documents. Like transformation-based learning (Brill, 1995) and data programming (Ratner et al., 2017; Smith et al., 2024), it produces readable rules, but an LLM writes them and measurement curates them.

Calibration and cascades. Neural and language models are often miscalibrated (Guo et al., 2017; Jiang et al., 2021), though large models can estimate their own correctness (Kadavath et al., 2022), and few-shot predictions can be recalibrated (Zhao et al., 2021). Selective prediction abstains on uncertain inputs (Kamath et al., 2020), and cascades escalate them to costlier models (Chen et al., 2024). We calibrate the *decision rules* of a program end to end against the task metric, and we cascade single closed questions, for which a stronger model's or a person's answer is a drop-in replacement.

Meaning representation parsing. Supervised AMR parsers (Bevilacqua et al., 2021; Zhou et al., 2021; Bai et al., 2022) train on tens of thousands of sentences (Knight et al., 2020), whereas UMR (Van Gysel et al., 2021; Bonn et al., 2023) has little gold data (Bonn et al., 2024, 2025). Chun and Xue (2024) build UMR from an AMR parser and conversion rules, and SETUP (Markle et al., 2025) parses English sentences into UMR. Prompted LLMs struggle with AMR and UMR conventions (Ettinger et al., 2023; Sun et al., 2024).

3 Decision Orchestration

Setting. An annotation scheme defines a structured annotation y for a document x; in UMR, one rooted graph per sentence plus document-level temporal, modal and coreference relations. In the cold start we have a prose guideline, the external resources it refers to (here PropBank frames; Palmer et al., 2005), an automatic scorer, and a handful of gold documents. We train no model weights. We build, and later learn, a procedure Π that maps x to y.

Skills, registry and execution. Π consists of **skills**, a **registry** and numeric parameters θ. A skill has an input schema, an output schema (its *emit*) and an implementation: code (a *rule node*) or a model call (a *decision node*). The registry declares *waves* that run in order; for UMR these are sentence, reentrancy and document. Within a wave, a *dispatcher* maps fields of a skill's emit to the next skill. An *episode* starts at the wave's start skill (e.g., with a whole sentence) and grows a tree of nodes until every branch ends in a leaf. Control flow is

thus a fixed function of typed decisions. The model never chooses which skill runs next, how many calls are made, or how the output is formatted.

Table 8 in Appendix D lists the UMR nodes. `sent_type` classifies a span and routes it to `discourse`, `predicate`, `np_phrase` or `special_entity`, or stops it as a leaf. Graph-producing skills emit a fragment whose unexpanded parts are typed placeholders (*open slots*), each dispatched back to `sent_type`. Out-of-span participants go to the rule node `antecedent`, and the rule node `coverage` re-dispatches every token no child covers. The reentrancy wave merges coreferring nodes by rule. The document wave calls `document` once per sentence with its graph and the five preceding ones. An output handler assembles and validates the PENMAN graphs.

Two kinds of decision nodes. A decision node has two implementations that produce the *same* emit, so they can be swapped node by node. In a **fragment decision**, an LLM receives the skill's instructions (the relevant part of the guideline, rewritten for a model), the span and the numbered sentence. It may call `get_example` for worked examples and `propbank_lookup` for frames, and it answers through the emit schema, for example with a type from a closed list or a graph fragment with placeholders. It never writes a whole graph. In a **closed decision**, the skill's code enumerates candidates from the input (the words of the span, the senses and roles of a verb, the allowed relations, the preceding mentions) and asks *closed questions*: yes/no, a choice among options that each carry a one-line description from the guideline, or a graded score. A decision model returns a distribution over the options and no text. The skill is a coroutine: it yields a round of questions, receives the distributions, may ask a follow-up round, and returns an emit built by rules. `predicate`, for instance, asks three rounds (Figure 1b): the head and whether it is an abstract predicate; its sense, aspect, modal strength and polarity; and the role of every content word (one of the sense's roles, a general role, "part of the predicate" or "none"). Rules read the distributions against thresholds θ. For example, a span with a comma goes to `discourse` when $p(\text{discourse}) \geq 0.2$, and a coreference link is kept when $p \geq 0.4$. Closed mode needs no model that can write structured text, and every decision comes with a probability.

Traces and attribution. Every node records its input, its decision (in closed mode, the questions and full distributions) and its emit. After scoring, the scorer's alignment of predicted to gold variables lets us attribute every evaluation unit (a concept, role, attribute or relation) to the node that produced its predicted side. A fine-grained judge scores 25 aspects separately, such as senses, aspect, core arguments, name tokens, reentrancy, and temporal and coreference relations (Appendix A). Summing the lost credit per skill shows where the procedure fails, and the same records are what reflection reads.

4 Learning the Procedure from a Handful of Documents

All learning here changes Π (text, code, thresholds, registry) and never a model. We measure a procedure on gold documents D by

$$J(\Pi; D) = \frac{1}{|D|} \sum_{d \in D} \tfrac{1}{2}\big(F_d + C_d\big), \qquad (1)$$

where F_d is the sentence-graph F-score and C_d the comprehensive score that also covers the document level (Sun and Xue, 2024). Without C_d, changes to the document-level skill could not be measured. The few documents play different roles. Errors are diagnosed on *training* documents. *Regression* documents (earlier training documents) must not get worse. *Development* documents are measured but never diagnosed. Test documents are never seen.

4.1 Calibrating decision thresholds

Closed-mode rules read probabilities against thresholds that were set for one decision model. Another model spreads its probability mass differently, so a threshold rule fires too often or too rarely and the tree changes shape. What matters is the decision a rule takes, not each probability (cf. Zhao et al., 2021). We therefore fit θ to J directly. In coordinate descent, we sweep one threshold at a time over a few values, re-parse the calibration documents, and keep a value if it raises J by more than 0.05. Ten thresholds with three or four values each take about 40 parses of four documents, which is minutes with a decision model.

4.2 Routing uncertain decisions

A closed answer has a confidence $c = \max_o p(o)$. Questions with $c < \tau$ are asked again, with the same state and options, to a stronger answerer: a

frontier LLM forced to pick an option, or a person. The answer replaces the original, and nothing else in the procedure changes. In a cold start this lets annotators answer only the decisions the procedure is unsure of, instead of drawing graphs [E5].

4.3 Reflection

A round on a training document runs four steps with a strong LLM as the *reflector*. **Find**: for each sentence, given the gold and predicted graphs, the variable alignment, both document levels, the judge's failed units and the execution trace (with every question and its distribution), list each difference, why it is wrong, the responsible node, and the level of the cause (a wrong answer, a missing candidate or question, a wrong assembly rule, an upstream error, or doubtful gold). **Diagnose**: for each *place* (a decision skill, the shared library, or the structure), read its implementation and all its problems, and propose ranked changes with mechanism, evidence, expected effect and risk. **Edit**: turn each proposal into anchored edits (*modify*, *add* or *delete* at a unique anchor, *write* a file, or *tune* a constant over listed values). **Accept**: apply the edits to a copy of Π, which becomes the next, frozen version. Acceptance takes two forms.

Unmeasured (fragment mode). A frontier-LLM parse takes minutes and dollars, so an edit is accepted if its anchor matches exactly once and the skill still loads. Edits target instructions and examples. Rule nodes, registry and schemas are left alone, and the reflector is told to fix patterns, not instances. The next document, parsed before reflecting on it, tests the round.

Measured (closed mode). A decision model parses a document in seconds, so every edit is a controlled trial. Prioritized proposals are applied one at a time on top of those already accepted, and the training, regression and development documents are re-parsed after each. An edit is kept if J rises by at least 0.1 (or C by 0.15 for a document-level-only change), mean F and mean C each fall by at most 0.3, no document's F falls by more than $\max(0.8, 2\Delta J)$ (0.8 is about the run-to-run noise; the drop is capped at 2.5), and the regression documents fall by at most 0.3 on average. A rejected edit is sent back once with its measurement for a corrected attempt. Here the edit space is wide. Reflection may reword or add questions, add or prune options, split a long option list into a coarse and a fine question, change assembly rules, tune thresholds, add local resources, or rewire the registry. Measurement, not a restriction on what may change, keeps learning stable.

Guards. The reflector never sees test documents, and test scores come only from frozen versions. In closed mode, an edit is rejected if it contains six consecutive characters of any gold document, test documents included. In fragment mode this was a convention rather than an automatic check [TODO: run the check over the fragment-mode versions].

5 Experimental Setup

Data. We use the 31 Chinese UMR gold documents available to us (712 sentences; 8–82 per document), preprocessed as in Chun and Xue (2024). All scores are computed against the same gold revision.[1] Experiments were run in stages, so we report three test sets: T1 = 0004–0005 (99 sentences) and T2 = 0005–0007 (157) for the fragment-mode runs, and H = 0015–0018, 0021–0024 (138) for the closed-mode runs. No reflection or hand tuning ever saw H. Training documents come from 0001–0014, and the development documents are 0027–0029 (47 sentences). Moving every system onto H is pending [E1].

Procedures. The fragment seed R_0 was written by hand from the UMR guidelines. R_{xy} is the version after round y on training document x. The closed seed J_0 uses the same registry. Its rules and options were tuned by hand on 0001–0003, so we test it only on documents from 0004 on. J_1 and J_2 follow unmeasured reflection on 0001 and 0002. K_1–K_4 continue from $K_0 = J_2$ with measured reflection on 0008–0011, one document per round.

Models. Fragment decisions are made by Claude Sonnet 5.5, by DeepSeek V4.1 Flash (test runs without thinking), and by Qwen3.5-9B and Qwen3.8-27B (Yang et al., 2025) served with vLLM. Closed decisions are answered by three decision-model services that share one protocol: Jev (TypeSafe, 2026), the OpenAI Decisions API with `gpt-6-luna` (OpenAI, 2026), and Microsoft-Decision-1, a post-trained Qwen3.5-9B (Microsoft, 2026). The reflector is Claude Opus 5.5 unless the parsing model reflects on itself.

[1] Some runs were first scored against an earlier revision. Re-scoring every stored prediction moved no macro average by more than 0.3 F.

Baselines. *Single call*: one call per sentence with the sentence, a request for its UMR graph and a pointer to the guidelines (no document level). *Detailed single call*: a full description of the target format, plus a document-level call. Pending [E2]: a single call with the full guideline and gold examples from the same training documents; a generic tool-using agent; and the pipeline of Chun and Xue (2024).

Metrics. AnCast++ (Sun and Xue, 2024): sentence-graph F-score (F); modal, temporal and coreference scores; and the comprehensive score (Comp.). Each configuration was run once. A cold re-run changes F by up to 0.6 in closed mode and by 2.3 on average for Qwen3.5-9B in fragment mode, so we treat differences below about 1 F as noise. Repeated runs are pending [E3].

6 Results

6.1 Orchestration versus generating the annotation

Table 1 gives the main results. Asked for a whole graph, Sonnet 5.5 reaches 29–33 F and writes no document level. The hand-written procedure, with the same model making only local decisions, starts at 45.3 F on T1 and adds the document level (Comp. 44.0 vs. 27.7). For a 9B model, orchestration makes the difference between nothing and something. Single calls yield no scorable graph on any of 24 documents, and a detailed single call reaches 4.9 F on H. Inside the procedure the same model reaches 17.6 F, ahead of the detailed call on all 21 documents where both were run. Output is well-formed by construction: the judge's format check is $\geq$0.996 in every Sonnet run. Whether a generic agent with the same guideline and tools closes the gap, and at what cost, is pending [E2].

6.2 Who answers the decisions

Because the decision interface is fixed, very different models can serve the same procedure (Table 2). In fragment mode, Qwen3.8-27B reaches 42.4 F on 0001–0004, 94% of Sonnet's 45.1. DeepSeek V4.1 Flash depends on its reasoning: on 0002 it scores 49.1 F with thinking and 35.9 without (Sonnet: 50.6), so its no-thinking test runs in Table 1 understate it. In closed mode the model never writes structure, and a 9B decision model reaches 30.8 F on H, against 17.6 for a 9B model in fragment mode. A procedure tuned for one answerer does not transfer unchanged, though: K_0, built with Jev (39.5), loses 8.7–13.9 F with the other two decision models. Part of the gap is calibration (§6.4).

6.3 Learning from a handful of documents

Fragment mode. With Sonnet parsing and Opus reflecting, one training document (51 sentences) raises T1 from 45.3 to 50.8 F, and a second raises it to 52.6 (Table 1). Temporal relations improve most (29.5 $\rightarrow$ 43.6). A third and fourth document leave test scores flat. The first parse of each new document, with a version that never saw it, tells the same story: 0002 goes from 50.6 (seed) to 58.9 (R_{13}), and 0003 from 49.7 to 57.5 (R_{23}; trajectories in Appendix B). Weaker models can reflect on themselves. DeepSeek gains 5.2 F on T2 from two documents, and Qwen3.8-27B gains 11.1 F on its training document in one round. Qwen3.5-9B cannot (§7).

Closed mode. Two rounds of unmeasured reflection improve the Jev procedure on every test set: T1 36.8 $\rightarrow$ 40.2, T2 34.5 $\rightarrow$ 38.9, and 9 of the 10 unseen documents 0003–0012. Measured reflection then continues on new documents (Table 3), accepting 6–8 of 12 proposals per round. On H, K_1 and K_2 add 1.0 and 0.6 F, K_3 is flat and K_4 loses 0.7, while the training documents gain an order of magnitude more (+4.1 to +16.8; see §7). Learning curves over several document orders are pending [E3].

6.4 Calibration and routing

Thresholds do not transfer; recalibration is cheap. K_0's thresholds were tuned with Jev. Answered by the OpenAI Decisions API, it scores 25.6 F on H. Coordinate descent on four gold documents (0001–0003, 0006) finds that only two of ten thresholds matter: when a span counts as a single-verb clause (0.6 $\rightarrow$ 0.7) and the floors for attaching an out-of-clause argument (0.4/0.6 $\rightarrow$ 0.3/0.5). The calibration objective rises from 29.0 to 30.8, and on H, which calibration never saw, F rises from 25.6 to 28.5 (Comp. 27.0 $\rightarrow$ 29.6). No value of the other eight improves the objective by more than 0.05 (Appendix C). The new model is miscalibrated, relative to the procedure, only where a threshold decides the shape of the tree. Calibrating Microsoft-Decision-1, and calibrating from one or two documents, are pending [E4].

Confidence predicts agreement. On J_2's questions for two unseen documents (0010, 0012), Jev and Sonnet agree on 95% of the questions where

System	Decisions by	Gold docs (sent.)	T1: 0004–05 F/ Comp.	T2: 0005–07 F/ Comp.	H: 0015–24 F/ Comp.
Baselines: one call per sentence					
Single call	Sonnet 5.5	0	32.7 / 27.7	29.4 / 24.9	–
Single call	Qwen3.5-9B	0	–	–	0.0 / 0.0
Detailed single call	Qwen3.5-9B	0	–	–	4.9 / 5.3
Full guideline + gold examples [E2]	Sonnet 5.5	4 (201)	–	–	–
Generic tool-using agent [E2]	Sonnet 5.5	0	–	–	–
Fragment-mode orchestration (unmeasured reflection by Opus 5.5)					
R_0 (seed)	Sonnet 5.5	0	45.3 / 44.0	–	–
R_{13}	Sonnet 5.5	1 (51)	50.8 / 49.4	–	–
R_{23}	Sonnet 5.5	2 (111)	**52.6 / 51.2**	–	–
R_{32}	Sonnet 5.5	3 (142)	52.6 / 51.1	55.2 / 52.1	–
R_{42}	Sonnet 5.5	4 (201)	n/a	**55.6 / 52.2**	–
R_0 (seed)	DeepSeek V4.1‡	0	–	30.8 / 29.0	–
R_{42} (self-reflection)	DeepSeek V4.1‡	2 (110)	–	36.0 / 34.0	–
R_0 (seed)	Qwen3.5-9B	0	–	–	17.6 / 18.5
Closed-mode orchestration					
J_0 (seed)	Jev	0†	36.8 / 37.1	34.5 / 33.8	–
$J_2 = K_0$ (unmeasured reflection)	Jev	2 (111)	40.2 / 41.3	38.9 / 38.0	39.5 / 38.4
K_2 (measured reflection)	Jev	4 (131)	–	–	**41.1 / 39.8**
K_4 (measured reflection)	Jev	6 (168)	–	–	40.1 / 39.2

Table 1: Main results on Chinese UMR (AnCast++, macro average over documents; F = sentence-graph F, Comp. = comprehensive score including the document level). "Gold docs" counts the training documents the procedure learned from. "n/a": 0004 was a training document. †J_0's rules were tuned by hand on 0001–0003. ‡Parsed without thinking (§6.2). Red dashes are pending experiments ([E1], [E2]); "–" marks a cell not planned.

Answerer	Mode	F	Comp.
Qwen3.5-9B	fragment, R_0	17.6	18.5
Qwen3.8-27B	fragment, R_0	–	–
Sonnet 5.5	fragment, R_0	–	–
Microsoft-Decision-1 (9B)	closed, K_0	30.8	31.6
OpenAI Decisions	closed, K_0	25.6	27.0
+ calibrated (§6.4)	closed, K_0^θ	28.5	29.6
Jev	closed, K_0	39.5	38.4
Sonnet 5.5	closed, K_0	–	–

Table 2: One procedure, different answerers, on H. Pending: [E1], [E5].

Round (doc, sent.)	acc./tried	train F	reg. ΔF	H F
K_0				39.5
K_1 (0008, 10)	8/12	40.0→54.1	+0.5	40.5
K_2 (0009, 10)	7/12	43.4→60.2	+0.5	**41.1**
K_3 (0010, 12)	8/12	41.4→45.9	+0.4	40.9
K_4 (0011, 25)*	6/12	37.5→41.6	+3.2	40.1

Table 3: Measured reflection with Jev. "reg. ΔF": mean change on the regression documents. *From round 4 on, problems are diagnosed on the training and regression documents together, and three development documents are measured in every trial (mean +0.8 F on them).

Jev's confidence is ≥0.9 (53% of questions), 79% at 0.7–0.9 (26%), and 57% below 0.7 (21%). Disagreement concentrates in `sent_type`'s routing question (78%) and `np_phrase`'s modifier questions (67%), where a wrong answer reshapes a subtree. Reliability against gold-derived answers is pending [E4].

Routing. Routing every question with confidence below $\tau = 0.7$ from Jev to Sonnet (Table 4) recovers 3.3 of the 4.4 F that Sonnet gains by answering everything (75%), with 39% of its calls and 26% of its cost. The curve over τ, and a cold-start variant in which routed questions are answered from gold as a stand-in for an annotator, are pending [E5].

6.5 A second benchmark

[TODO: E9, to be run.] We will test whether the recipe transfers beyond UMR on verbal multiword expression identification in PARSEME 1.2 (Ramisch et al., 2020), whose guidelines are almost literally decision trees of linguistic tests, and whose test sets center on about 300 unseen expressions per language, on which the best participating system reaches only 38.5 F1 (Appendix F).

7 Analysis

Errors are attributable to nodes. Lost credit can be summed per skill (Table 6, Appendix B). On 0001, reflection reduces losses most where a fix is local. Lost credit in `special_entity` falls

Answerer	F	Sonnet calls	Cost
Jev only	39.1	0	<$0.1
Jev; $c < 0.7$ to Sonnet	42.4	504	≈$9
Sonnet only	43.5	1,292	≈$34
other τ, on H [E5]	–	–	–
$c < \tau$ from gold [E5]	–	–	–

Table 4: Routing uncertain closed decisions of J_2 on 0010+0012 (20 sentences). Questions and rules are the same in every row; only the answerer changes. Public API prices.

from 149 to 33 and in `np_phrase` from 406 to 215, while name tokens rise from 0.38 to 0.87 F and the core arguments of abstract predicates from 0.13 to 0.75. Losses that span several decisions improve least: `document` only from 406 to 337, reentrancy from 0.15 to 0.36, coreference chains from 0.36 to 0.44. The "no predicted node" row, where an upstream routing error left nothing to align, stays above 200 and measures the cost of the cascade. [TODO: recompute the judge numbers against the current gold revision]

Swapping one component locates the bottleneck. Questions, rules and answerers are separate, so each can be replaced while the rest stays fixed. On 0010 and 0012 with J_2's questions and rules, answering everything with Sonnet instead of Jev raises F from 39.1 to 43.5. Segmenting words at gold concept boundaries adds 1.0 (1.5 on average over 12 documents). The judgments are thus worth about 4 F. Yet a frontier model answering the same questions stops at 43.5, below the fragment-mode seed with that model (45.3 on T1, other documents), so after two rounds the bottleneck lay in candidates, rules and tree shape, not in judgment. This is why measured reflection may change rules and structure. Of the 29 edits it accepted, 22 change assembly rules, 4 option sets, 1 adds a tool, 1 a question, and 1 changes wording. The earlier unmeasured rounds had mostly reworded option descriptions.

Measurement turns reflection into experiments. Unmeasured reflection is not filtered. In fragment mode each round finds 376–902 problems (29–43% at the document level) and applies 35–73 edits, and only 1 of 441 edits is rejected. The six skills grow from 41k to 156k characters over ten rounds, the same accumulation of rules that makes hand-maintained guidelines hard to use. With measurement, 19 of 48 proposals in four rounds were reverted, among them third-round attempts to *undo* earlier fixes, which the regression documents rejected. A retry with feedback often rescues an edit; one edit that first dropped a document by 2.95 F was rewritten into one that raised the objective by 1.43 with no document falling.

Why held-out gains are an order of magnitude smaller. In rounds 1–3 of measured reflection, the training document gains 4.5–16.8 F and H gains −0.2 to 1.0. On H, the judge aspects fixed on training documents barely move from K_0 to K_3 (entity concepts 0.646 → 0.661, event core arguments 0.385 → 0.399). Many fixes encode one document's vocabulary and conventions, such as currency denominations, product names, or the imperatives of an instruction text. They are correct but orthogonal to other texts. Round 4 diagnosed across training and regression documents and measured three development documents. The regression documents gained 1.9–5.4 and the development documents 0.8 on average, yet H lost 0.7, with drops of 1.8 and 2.3 on two documents. This is not noise: cold re-runs of one version differ by ≤0.6, while K_3 and K_4 differ by 1.7–3.5 on those documents. With a handful of documents, the *acceptance test* limits generalization, since 47 development sentences of one genre cannot vouch for an edit. Ablating round 4 and enlarging the development set are pending [E6]. [TODO: report the ablation]

Weak reflectors and cost. With Qwen3.5-9B as parser and reflector, each new document parsed with the previous version is 1.4 F *below* the seed on average (noise: 2.3). The 9B reflector rewrites the anchors it should copy (1 of 15 edits applied) and writes noisy rules, so attribution alone is not enough. As for cost, fragment mode with Sonnet takes dozens of calls per sentence ($0.71 per sentence on T1), whereas closed mode took 7,552 requests and about $0.64 for 216 sentences, and four rounds of measured reflection about $8 in decision requests plus 36–82 reflector calls per round [E11].

8 Conclusion

With a guideline and a handful of gold documents, the thing to learn is the procedure, not a model. Decision orchestration lets code own control flow and well-formedness and consults a model only for local decisions. Small models can then annotate a rich scheme, and the procedure can be calibrated,

routed and corrected from a few documents through errors attributed to single nodes. The same explicitness shows the limit: with so few documents, the acceptance test, not the reflector, bounds how far learned edits generalize. [TODO: second benchmark (E9) and human routing (E5)]

Limitations

Our main evidence comes from one task in one language. The second benchmark (§6.5) has not been run yet, and the English UMR runs, made with earlier versions of the procedure, are not reported. Test sets are small (99–157 sentences) and differ between stages of the work. Unifying them is planned ([E1]). Most configurations were run once, and run-to-run noise (up to 0.6 F for closed mode and 2.3 F for a 9B model in fragment mode) is of the same order as some of the differences we report. The seeds were written by the authors, so their quality, and the effort it took to write them, affect every result. We have not measured that effort. Several of the models we use, including all three decision-model services and the frontier LLMs, are commercial APIs whose behavior may change; we release the cached answers to make runs reproducible [TODO: check that the release includes them]. The routing results come from 20 sentences, and the "human" in routing will be simulated from gold, which overstates real annotators' accuracy. Finally, AnCast and our fine-grained judge depend on an automatic alignment, and the gold itself contains errors. One gold graph has unbalanced brackets and is skipped by the scorer.

Ethics Statement

The data are publicly released Chinese UMR annotations. We collected no new data from people and no personal information beyond what the released documents contain. Our goal is to reduce the cost of producing the first annotations for a new scheme, which could reduce demand for annotation work. We see the method as a way to direct annotators' effort to the decisions that need human judgment. Running the experiments in this paper used commercial APIs, at a total cost we will report with the camera-ready version [TODO: sum the API costs].

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*.

Xuefeng Bai, Yulong Chen, and Yue Zhang. 2022. Graph pre-training for AMR parsing and generation. *arXiv preprint arXiv:2203.07836*.

Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969.

Michele Bevilacqua, Rexhina Blloshmi, and Roberto Navigli. 2021. One SPRING to rule them both: Symmetric AMR semantic parsing and generation without a complex pipeline. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(14):12564–12573.

Julia Bonn, Claire Bonial, Matt Buchholz, Hsiao-Jung Cheng, Alvin Chen, Ching-wen Chen, Andrew Cowell, William Croft, Lukas Denk, Ahmed Elsayed, Eva Fučíková, Federica Gamba, Carlos Gomez, Jan Hajič, Eva Hajičová, Jiří Havelka, Loden Havenmeier, Ath Kilgore, Veronika Kolářová, and 40 others. 2025. Uniform meaning representation 2.0.

Julia Bonn, Matthew J. Buchholz, Jayeol Chun, Andrew Cowell, William Croft, Lukas Denk, Sijia Ge, Jan Hajič, Kenneth Lai, James H. Martin, Skatje Myers, Alexis Palmer, Martha Palmer, Claire Benet Post, James Pustejovsky, Kristine Stenzel, Haibo Sun, Zdeňka Urešová, Rosa Vallejos, and 4 others. 2024. Building a broad infrastructure for uniform meaning representations. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 2537–2547, Torino, Italia. ELRA and ICCL.

Julia Bonn, Chen Ching-wen, James Andrew Cowell, William Croft, Lukas Denk, Jan Hajič, Kenneth Lai, Martha Palmer, Alexis Palmer, James Pustejovsky, Haibo Sun, Rosa Vallejos Yopán, Jens Van Gysel, Meagan Vigus, Nianwen Xue, and Jin Zhao. 2023. Uniform meaning representation. LINDAT/CLARIAH-CZ digital library at the Institute of Formal and Applied Linguistics.

Eric Brill. 1995. Transformation-based error-driven learning and natural language processing: A case study in part-of-speech tagging. *Computational Linguistics*, 21(4):543–565.

Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*.

Jayeol Chun and Nianwen Xue. 2024. A pipeline approach for parsing documents into uniform meaning representation graphs. In *Proceedings of the Fifth*

International Workshop on Designing Meaning Representations @ LREC-COLING 2024.

Allyson Ettinger, Jena D. Hwang, Valentina Pyatkin, Chandra Bhagavatula, and Yejin Choi. 2023. "you are an expert linguistic annotator": Limits of llms as analyzers of abstract meaning representation. *arXiv preprint arXiv:2310.17793*.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799. PMLR.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, Singapore. Association for Computational Linguistics.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR.

Tanmay Gupta and Aniruddha Kembhavi. 2023. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14953–14962.

Xingwei He, Zhenghao Lin, Yeyun Gong, A-Long Jin, Hang Zhang, Chen Lin, Jian Jiao, Siu Ming Yiu, Nan Duan, and Weizhu Chen. 2024. AnnoLLM: Making large language models to be better crowdsourced annotators. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 165–190, Mexico City, Mexico. Association for Computational Linguistics.

Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*.

Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. 2021. How can we know when language models know? on the calibration of language models for question answering. *Transactions of the Association for Computational Linguistics*, 9:962–977.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, and 1 others. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.

Amita Kamath, Robin Jia, and Percy Liang. 2020. Selective question answering under domain shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5684–5696, Online. Association for Computational Linguistics.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, and 1 others. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*.

Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2023. Decomposed prompting: A modular approach for solving complex tasks. In *The Eleventh International Conference on Learning Representations*.

Kevin Knight, Bianca Badarau, Laura Baranescu, Claire Bonial, Madalina Bardocz, Kira Griffitt, Ulf Hermjakob, Daniel Marcu, Martha Palmer, Tim O'Gorman, and Nathan Schneider. 2020. Abstract Meaning Representation (AMR) Annotation Release 3.0. LDC2020T02.

Minzhi Li, Taiwei Shi, Caleb Ziems, Min-Yen Kan, Nancy Chen, Zhengyuan Liu, and Diyi Yang. 2023. CoAnnotating: Uncertainty-guided work allocation between human and large language models for data annotation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1487–1505, Singapore. Association for Computational Linguistics.

Aman Madaan, Niket Tandon, Prakhar Gupta, and 1 others. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*.

Emma Markle, Javier Gutierrez Bach, and Shira Wein. 2025. Setup: Sentence-level english-to-uniform meaning representation parser. *arXiv preprint arXiv:2512.07068*.

Microsoft. 2026. Use Microsoft-Decision-1 in Azure AI Foundry. `https://learn.microsoft.com/en-us/azure/foundry/foundry-models/how-to/use-foundry-models-microsoft-decision`. Accessed October 2026.

OpenAI. 2026. Decisions API guide. `https://developers.openai.com/api/docs/guides/decisions`. Accessed October 2026.

Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, Miami, Florida, USA. Association for Computational Linguistics.

Martha Palmer, Daniel Gildea, and Paul Kingsbury. 2005. The proposition bank: An annotated corpus of semantic roles. *Computational Linguistics*, 31(1):71–106.

Chaoxu Pang, Yixuan Cao, Qiang Ding, and Ping Luo. 2023. Guideline learning for in-context information extraction. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15372–15389, Singapore. Association for Computational Linguistics.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.

Carlos Ramisch, Agata Savary, Bruno Guillaume, Jakub Waszczuk, Marie Candito, Ashwini Vaidya, Verginica Barbu Mititelu, Archna Bhatia, Uxoa Iñurrieta, Voula Giouli, Tunga Güngör, Menghan Jiang, Timm Lichte, Chaya Liebeskind, Johanna Monti, Renata Ramisch, Sara Stymne, Abigail Walsh, and Hongzhi Xu. 2020. Edition 1.2 of the PARSEME shared task on semi-supervised identification of verbal multiword expressions. In *Proceedings of the Joint Workshop on Multiword Expressions and Electronic Lexicons*, pages 107–118. Association for Computational Linguistics.

Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. 2017. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282.

Oscar Sainz, Iker García-Ferrero, Rodrigo Agerri, Oier Lopez de Lacalle, German Rigau, and Eneko Agirre. 2024. GoLLIE: Annotation guidelines improve zero-shot information-extraction. In *The Twelfth International Conference on Learning Representations*.

Manon Scholivet, Agata Savary, Carlos Ramisch, Eric Bilinski, Takuya Nakamura, Maria Mitrofan, and Vasile Pais. 2026. Edition 2.0 of the PARSEME shared task on multilingual identification and paraphrasing of multiword expressions. In *Proceedings of the 22nd Workshop on Multiword Expressions (MWE 2026)*, pages 254–275. Association for Computational Linguistics.

Burr Settles. 2009. Active learning literature survey. Computer Sciences Technical Report 1648, University of Wisconsin–Madison.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*.

Ryan Smith, Jason A. Fries, Braden Hancock, and Stephen H. Bach. 2024. Language models in the loop: Incorporating prompting into weak supervision. *ACM / IMS Journal of Data Science*, 1(2):1–30.

Haibo Sun and Nianwen Xue. 2024. Anchor and broadcast: An efficient concept alignment approach for evaluation of semantic graphs. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 1052–1062, Torino, Italia. ELRA and ICCL.

Haibo Sun, Nianwen Xue, Jin Zhao, Liulu Yue, Yao Sun, Keer Xu, and Jiawei Wu. 2024. Chinese UMR annotation: Can LLMs help? In *Proceedings of the Fifth International Workshop on Designing Meaning Representations @ LREC-COLING 2024*, pages 131–139.

Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. 2024. Let me speak freely? a study on the impact of format restrictions on large language model performance. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1218–1236, Miami, Florida, US. Association for Computational Linguistics.

Zhen Tan, Dawei Li, Song Wang, Alimohammad Beigi, Bohan Jiang, Amrita Bhattacharjee, Mansooreh Karami, Jundong Li, Lu Cheng, and Huan Liu. 2024. Large language models for data annotation and synthesis: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 930–957, Miami, Florida, USA. Association for Computational Linguistics.

TypeSafe. 2026. Jev API reference. `https://docs.typesafe.ai/api`. Accessed October 2026.

Jens E. L. Van Gysel, Meagan Vigus, Jayeol Chun, Kenneth Lai, Sarah Moeller, Jiarui Yao, Tim O'Gorman, Andrew Cowell, William Croft, Chu-Ren Huang, Jan Hajič, James H. Martin, Stephan Oepen, Martha Palmer, James Pustejovsky, Rosa Vallejos, and Nianwen Xue. 2021. Designing a uniform meaning representation for natural language processing. *KI - Künstliche Intelligenz*, 35(0):343–360.

Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. 2021. Want to reduce labeling cost? GPT-3 can help. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4195–4205, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Xingyao Wang, Sha Li, and Heng Ji. 2023. Code4Struct: Code generation for few-shot event structure prediction. In *Proceedings of the 61st Annual Meeting of*

the Association for Computational Linguistics (Volume 1: Long Papers), pages 3640–3663, Toronto, Canada. Association for Computational Linguistics.

Xinru Wang, Hannah Kim, Sajjadur Rahman, Kushan Mitra, and Zhengjie Miao. 2024. Human-LLM collaborative annotation through effective verification of LLM labels. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*, pages 1–21, Honolulu, HI, USA. ACM.

Brandon T. Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.

Tongshuang Wu, Michael Terry, and Carrie Jun Cai. 2022. AI chains: Transparent and controllable human-AI interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*, pages 1–22, New Orleans, LA, USA. ACM.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *International Conference on Learning Representations*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by backpropagating language model feedback. *Nature*.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2024. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling*.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19632–19642.

Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. 2021. Calibrate before use: Improving few-shot performance of language models. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 12697–12706. PMLR.

Jiawei Zhou, Tahira Naseem, Ramón Fernandez Astudillo, Young-Suk Lee, Radu Florian, and Salim Roukos. 2021. Structure-aware fine-tuning of sequence-to-sequence transformers for transition-based AMR parsing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6279–6290, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large language models are human-level prompt engineers. In *International Conference on Learning Representations*.

Zhaocheng Zhu, Yuan Xue, Xinyun Chen, Denny Zhou, Jian Tang, Dale Schuurmans, and Hanjun Dai. 2023. Large language models can learn rules. *arXiv preprint arXiv:2310.07064*.

Caleb Ziems, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang. 2024. Can large language models transform computational social science? *Computational Linguistics*, 50(1):237–291.

A Fine-grained judge

The judge works on the AnCast alignment of predicted and gold variables. It scores each of the following checks as an F-score over *units*. A unit is one small comparison (a node, an attribute, an argument set, a relation) that is correct, partially correct, wrong, missing or extra. **(a) nodes**: event found, event concept, entity found, entity concept, granularity (is the node cut at the same words as gold); **(b)** event sense; **(c) attributes**: aspect, modal strength, polarity, refer-person/number, other attributes; **(d) arguments**: core arguments of events, core arguments of abstract predicates, peripheral arguments of events, peripheral arguments of entities; **(e) role labels**: core, peripheral; **(f) names**: name tokens, wiki; **(g)** reentrancy; **(h)** temporal; **(i)** modal; **(j)** coreference chains and subsets; **(k)** document format. For attribution, each unit is assigned

to the node that produced its predicted side, or to "no predicted node" when only the gold side exists.

B Training trajectories

Doc	seed	iter 1	iter 2	iter 3	after
0001	40.3	40.3	59.6	66.9	70.9
0002	50.6	58.9	66.3	70.4	73.4
0003	49.7	57.5	69.5	–	–
0004	39.6	49.7	57.9	–	–

Table 5: Fragment mode (Sonnet 5.5 parsing, Opus 5.5 reflecting): sentence F on each training document. Iteration y parses with the version before round y, so iteration 1 uses only earlier documents. "after" parses with the document's final version.

Skill	iter 1	iter 2	iter 3	after
`predicate`	518	435	440	395
`np_phrase`	406	361	266	215
`document`	406	380	338	337
(no predicted node)	286	295	259	211
`sent_type`	273	180	145	148
`special_entity`	149	62	38	33
`discourse`	26	28	31	37

Table 6: Credit lost per skill on training document 0001 over three rounds of fragment-mode reflection (sum of $1 -$ credit over all judge units attributed to the skill). [TODO: recompute the judge numbers in this paragraph against the current gold revision; AnCast scores already are.]

C Threshold calibration

Table 7 lists the thresholds of the closed-mode procedure and the sweep for the OpenAI Decisions API. The objective is Eq. 1 on 0001–0003 and 0006. Thresholds were swept in the order listed, each on top of the values kept so far, and a value is kept if it improves the objective by more than 0.05.

Threshold (skill)	Jev	tried	kept (ΔJ)
comma span → discourse (`sent_type`)	.2	.3 .4 .5	.2
split noun phrase (`sent_type`)	.2	.3 .4 .5	.2
word is an event (`sent_type`)	.5	.4 .6 .7	.5
single-verb clause (`sent_type`)	.6	.4 .5 .7	**.7** (+1.55)
conjunct cut, and/or (`discourse`)	.4/.5	+.1, +.2, +.3	.4/.5
cut without comma (`discourse`)	.5	.6 .7 .8	.5
outside argument, ARG0-1/other (`predicate`)	.4/.6	±.1, +.2	**.3/.5** (+0.27)
generic yes/no (shared)	.5	.6 .7 .8	.5
coreference kept (`document`)	.4	.3 .5 .6	.4
modal source kept (`document`)	.7	.5 .6 .8	.7

Table 7: Calibration of K_0 for the OpenAI Decisions API. Objective 29.0 → 30.8 on the calibration documents; on H, F 25.6 → 28.5. [TODO: same table for Microsoft-Decision-1 (E4)]

D Skills

Node	Kind	Decides or does
`sent_type`	decision	span type (5-way); routing
`discourse`	decision	relation; split points
`predicate`	decision	head, sense, roles, aspect, modality
`np_phrase`	decision	head, its form, modifier roles
`special_entity`	decision	entity kind, type, value
`document`	decision	time anchor, source, coreference
`antecedent`	rule	out-of-span participants
`coverage`	rule	re-dispatch uncovered tokens
`reentrancy`	rule	merge same-referent nodes
output handler	rule	assemble, validate

Table 8: Nodes of the UMR procedure. Each decision node has two interchangeable implementations (fragment and closed); rule nodes, registry and scorer are shared.

A fragment-mode skill. `predicate/skill.md` has sections on the head concept (event, abstract predicate, sense), the participants, participants outside the clause, aspect, modal strength and polarity, the output, worked examples, and, last, an `## Input` section with the span, the numbered sentence and the outputs of the parent node. Because the input comes last, everything before it is cached across calls. Reflection edits these sections in place by anchored edits. The emit schema lists closed value sets for aspect (27 values), modal strength (6) and polarity, and open fields for the fragment, its open slots and its out-of-span participants. Over ten rounds of unmeasured reflection, the six decision skills grow from 41k to 156k characters of instructions.

A closed-mode skill. A closed skill is a Python generator:

```
def run(sent, span, **_):
    a = yield {"state": {...}, "questions": {
        "type": {"type": "choice",
                 "criteria": TYPES}}}
    b = yield {...}   # may depend on a
    return emit_from(a, b)  # rules
```

Each option in `TYPES` carries a description condensed from the guideline, for example what counts as a *discourse* span in Chinese, with the constructions that do and do not qualify. The returned emit has the same schema as the fragment-mode skill, so the registry and rule nodes are unchanged. Table 9 lists the questions per skill.

Skill	Rounds of closed questions
`sent_type`	span type (5-way); for a one-word span in PropBank, its use here (event, state, manner, thing)
`discourse`	relation (8 frequent ones); one yes/no per possible cut point
`predicate`	(1) head word, verb or abstract predicate; (2) sense, aspect, modal strength, negation; (3) role of each content word; antecedent of each core argument outside the clause
`np_phrase`	(1) head and its form (8-way); (2) role of each modifier; sense if the head is an event
`special_entity`	kind (name, date, quantity, percentage, ordinal); entity type (44); quantity type
`document`	(1) temporal relation to the anchor event; source of a view; earlier mention it co-refers with, or new; (2) confirm each co-reference (yes/no)

Table 9: Closed questions asked by each decision skill of J_0. Values (numbers, dates, names) are read by rules.

E Reflection prompts

[TODO: condense rsi/find_problems.md, rsi/plan_edits.md and rsi-jev2/diagnose.md into half a page each; include the list of proposal kinds and the acceptance constants of §4.3]

F Second benchmark: planned procedure

Task and data. PARSEME 1.2 (Ramisch et al., 2020): identify verbal multiword expressions and their category in sentences in the `.cupt` format (CoNLL-U plus an MWE column), Chinese and one more language. We will simulate the cold start by drawing "documents" of 100 training sentences and learning from 1, 2 and 4 of them, measuring on the official development set and testing once on the official test set. Metrics are the official MWE-based and token-based P/R/F and MWE-based F on unseen expressions.

Procedure. Candidates (rule). For each verb, enumerate the combinations with its dependents that a category allows: a verb with a nominal object or oblique (light-verb construction); a verb with a particle or, in Chinese, a resultative or directional complement (verb-particle construction); serial verbs (multi-verb construction, following the Chinese-specific decision tree); and a verb with lexicalized dependents (verbal idiom). Expressions seen in training are matched by lemma (rule). **Decisions (closed).** Each guideline test becomes a yes/no question about one candidate in context, for example whether the noun is predicative, whether the verb is semantically light, or whether replacing a component changes the meaning. The decision tree of the guidelines becomes code that asks the next test according to the answers. **Assembly (rule).** Spans and categories are written to `.cupt`. Overlapping candidates are resolved by rule. **Learning.** Thresholds are calibrated per test. Measured reflection may reword tests, add candidate patterns or reorder the tree.

Why PARSEME. The PARSEME guidelines are, almost literally, a decision procedure. Candidates are found on the dependency trees that ship with the data, and each candidate is classified by linguistic tests organized as decision trees into a closed set of categories (verbal idioms, light-verb constructions, inherently reflexive verbs, verb-particle and multi-verb constructions). The test sets contain about 300 expressions per language that are unseen in training, which is the regime a procedure learned from a few documents should help with. The data is freely available, has an official scorer, and includes Chinese, which keeps the language constant with our UMR experiments; we will add one language from a different family [TODO: choose: French or Polish]. Edition 2.0 extends the scheme to all syntactic categories in 17 languages (Scholivet et al., 2026), which allows a second test after a schema revision. Table 10 lists the reference numbers.

PARSEME 1.2, Chinese test	unseen F1	global F1
Two-rule baseline (Ramisch et al., 2020)	27.3	–
Best 1.2 system on ZH (TRAVIS-mono)	60.2	–
Single LLM call with guidelines [E9]	–	–
Orchestration, seed [E9]	–	–
+ 1/2/4 training documents [E9]	–	–

Table 10: Planned second benchmark. Reference numbers are from Ramisch et al. (2020). Global F1 for the reference systems will be taken from the official result tables.

Baselines. The two-rule baseline and the official results of edition 1.2; a single LLM call with the guidelines (zero-shot and with the same training sentences as examples); and the fragment-mode variant of the same procedure.

G Pending experiments (to be removed before submission)

E1 One gold revision, one held-out set (H), one development set for every system in Table 1.

E2 Stronger baselines: full guideline with gold

examples from the same training documents; a generic tool-using agent; Chun and Xue (2024).

E3 Learning curves over 0/1/2/4/8 documents with three document orders.

E4 Calibration: reliability diagrams and ECE against gold-derived answers for three decision models; calibrating Microsoft-Decision-1; calibration from 1, 2 and 4 documents; transfer of thresholds between models.

E5 Routing: the full curve over τ on H; routed questions answered from gold as a simulated annotator; Sonnet as the only closed answerer on H.

E6 Measured reflection: ablation of round 4, a larger and more varied development set, rounds 5–7, and the same loop with the other two decision models.

E7 Ablations of the procedure: without the coverage and reentrancy rules; whole-graph generation instead of fragments; without the PropBank tool.

E8 Ablations of reflection: wording-only edits vs. edits to rules, options and structure; unmeasured vs. measured acceptance at equal budget; training-only vs. training + regression + development measurement.

E9 Second benchmark (PARSEME 1.2, Appendix F).

E10 English UMR with the same procedure.

E11 Cost table: calls, tokens, dollars and wall-clock per configuration.

E12 Annotator time: correcting the procedure's output vs. annotating from scratch.